

Classical Image Restoration for UAV Imaging

항공전자정보공학부

Donghyeon Kim

ID: 2019124018

Korea Aerospace University

Abstract

This project explores the effectiveness of classical image processing techniques for image denoising and enhancement. Using knowledge gained from the 'Image Processing' course, we applied various filters—including median, box, Gaussian, and bilateral—to restore images corrupted by Gaussian and impulse noise. Median filters were most effective for impulse noise, while bilateral filters excelled for Gaussian noise, achieving up to 33 dB in PSNR.

For enhancement, we simulated a UAV imaging scenario with storage constraints, reconstructing full-resolution images from sparse observations at different altitudes. We proposed bicubic and forward interpolation methods, along with region selection strategies based on frequency energy, Laplacian residuals, and gradient magnitude. These approaches yielded up to 2.25 dB improvements over the baseline, demonstrating the potential of well-designed traditional methods in resource-limited settings.

1. Introduction

In the era of high-resolution imaging and real-time data acquisition, the importance of efficient image restoration techniques continues to grow. Denoising and enhancement algorithms play a crucial role in applications such as medical imaging, satellite surveillance, autonomous navigation, and unmanned aerial vehicles (UAVs), where degraded or incomplete data can hinder downstream tasks. Although deep learning-based methods have recently gained popularity due to their impressive performance, they often require large computational resources, complex architectures, and large-scale datasets—making them

less ideal for embedded or resource-constrained environments.

To address this, our project investigates how traditional image processing techniques can be leveraged to achieve effective image restoration without relying on heavy learning-based models. The first part of the study focuses on image denoising by simulating common noise types—Gaussian and impulse—and applying classical filtering methods to remove them. We quantitatively compare their effectiveness using Peak Signal-to-Noise Ratio (PSNR) and provide visual evidence for their strengths and limitations.

In the second part, we explore an image enhancement task inspired by UAV missions that collect images at varying altitudes and resolutions under strict memory constraints. We simulate this scenario and reconstruct high-resolution images using different interpolation techniques. Additionally, we introduce adaptive region selection methods that prioritize structurally informative areas based on frequency and spatial characteristics. By combining these techniques, we demonstrate that well-designed classical methods can deliver high-quality image restoration with minimal computational cost.

2. Problem 1: Image denoising.

1) Generate the following noises, and add them to the ground-truth images. Compute PSNR for noisy images.

-3.1.2 result

a) Gaussian noise (mean, var) = (0, σ^2)

-3.1.1 noise

b) Impulse noise

-3.1.1 noise

2) Denoise the generate noisy images using various filtering schemes. Compute PSNR for denoised images.

-3.2.2 result

3) Compare and analyze the results in terms of subjective and objective (PSNR) measurments.

-3.3 compare and analyze

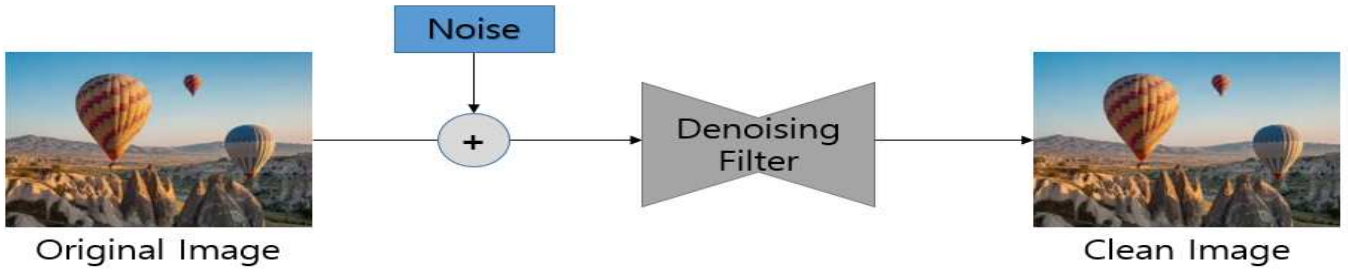


Figure 1. The diagram provides an overview of Problem 1. Gaussian noise and impulse noise are added to the image, and four different filters ‘median, box, Gaussian, and bilateral’ are applied for denoising.

3 Method: Image denoising

3.1.1 Noise

Gaussian noise

Gaussian noise is a type of statistical noise that follows a normal distribution, defined by a characteristic probability density function.

(μ : mean, σ : standard deviation)

$$P(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

In image processing, Gaussian noise is commonly used to simulate sensor or environmental noise. It affects all pixels independently by adding a random value sampled from a Gaussian distribution.

In our implementation, Gaussian noise was generated using the `np.random.normal(mean, std, image.shape)` function, which produces a noise matrix of the same shape as the input image. This noise is additively applied to the original image:

<Gaussian noise function>

```

def g_noise(image, mean, std):
    H, W, C = image.shape
    gauss = np.random.normal(mean, std,
                              image.shape)
    noisy = image + gauss
    noisy_image = np.clip(noisy, 0, 255).astype(
        np.uint8)

    return noisy_image
  
```

To prevent overflow or underflow, the resulting pixel values are clipped to the valid range [0,255] and converted back to 8-bit unsigned integers.

To evaluate the robustness of denoising algorithms under different noise conditions, we applied Gaussian noise at three standard deviation levels: 10, 30, and 50. The denoising performance was then measured using Peak Signal-to-Noise Ratio (PSNR), a widely used metric in image quality assessment.

Impulse noise

Impulse noise is characterized by the random replacement of pixel values with either 0 or 255, occurring independently with a probability $p/2$. It is often referred to as salt-and-pepper noise.

$$I'(x,y) = \begin{cases} 0, & p/2 \text{ (pepper)} \\ 255, & p/2 \text{ (salt)} \\ I(x,y), & 1-p \end{cases}$$

· $I(x,y)$: original pixel value

· $I'(x,y)$: noisy pixel value

Given a user-defined probability p , the total number of noisy pixels N is calculated as:

$$N = p * \text{total number of pixels}$$

To simulate impulse noise, we first determine the number of pixels to be corrupted based on the specified probability p . A one-dimensional array of indices ranging from 0 to the total number of pixels is generated using `np.arange(N_total)`, where $N=H \times W$, the height and width of the image. This array is then randomly shuffled using `np.random.shuffle` to ensure unbiased selection of pixels across the entire image. The first half of the shuffled indices are designated for pepper noise and are assigned a value of 0, while the remaining half are used for salt noise and set to 255. These 1D indices are mapped back to 2D spatial coordinates using the `np.unravel_index` function, allowing us to locate the exact (x, y) positions in the original image. Finally, the pixel values at these positions are overwritten with the corresponding salt or pepper values, effectively injecting high-contrast impulse noise into the image.

```

<Impulse noise function>
(...omitted)
num_noisy = int(prob * num_pixels)          # number of noisy pixels
all_coords = np.arange(num_pixels)          # 1D array
np.random.shuffle(all_coords)
salt_coords = all_coords[:num_noisy // 2]   # salt noise
pepper_coords = all_coords[num_noisy // 2:num_noisy] # pepper noise
salt_y, salt_x = np.unravel_index(salt_coords, (H, W))
pepper_y, pepper_x = np.unravel_index(pepper_coords, (H, W))
noisy[salt_y, salt_x, :] = 255
noisy[pepper_y, pepper_x, :] = 0

```

3.1.2 result

PSNR(dB)	Noisy Image				
	Impulse p = 0.05	Impulse p=0.1	Gaussian std = 10	Gaussian std = 30	Gaussian std = 50
	18.20	15.17	28.13	18.86	14.82

Table1. PSNR values for images corrupted with various types of noise. Max: ■ Min: ■



Figure 2. Visual comparison of original and noise-corrupted images using five common noise types.

3.2.1 Denoising

We designed a total of four image filters: Median Filter, Averaging Filter, Gaussian Filter, and Bilateral Filter. All filters were implemented as individual functions and applied with appropriate padding using 'np.pad', with the padding mode consistently set to 'reflect'. Reflect padding mirrors the pixel values at the image boundary, creating a reflection around the edge. This method preserves local continuity and reduces edge artifacts, making it well-suited for smoothing filters.

Median Filter

The Median Filter is a nonlinear denoising filter that replaces each pixel with the median of its local neighborhood $W(x,y)$. The filtered pixel $I'(x,y)$ is computed as:

$$I'(x,y) = \text{median}\{I(i,j) | (i,j) \in W(x,y)\}$$

It is particularly effective for removing impulse noise while preserving edges.

<Median filter function>

```
def median_filter(image, size):
    :
    padded = np.pad(image, ((pad, pad), (pad, pad),
                             (0, 0)), mode='reflect')

    for y in range(H):
        for x in range(W):
            for c in range(C):
                pixel_val = image[y, x, c]
                # filtering condition
                if pixel_val == 0 or pixel_val == 255:
                    window = padded[y:y+size, x:x+size,
                                    c]

                    filtered[y, x, c] = np.median(window)
                :
```

The function takes two inputs: (image, size), where size denotes the size of the sliding window applied to the image. We applied three nested for-loops to extract the median value for each color channel separately.

To improve performance, filtering was performed only when the pixel value was 0 or 255.

Averaging Filter(Mean Filter)

The Averaging Filter is also a linear smoothing filter that replaces each pixel value with the average of its neighboring pixels within a defined window.

Mathematically, for a window $W(x,y)$ of size $k \times k$, the filter pixel $I'(x,y)$ is computed as:

$$I'(x,y) = \frac{1}{k^2} \sum_{(i,j) \in W(x,y)} I(i,j)$$

This filter is effective for reducing random noise, but it tends to blur edges and fine details due to uniform smoothing.

<Averaging filter function>

```
def averaging_filter(image, size):
    :
    # kernel generation
    kernel = np.ones((size, size)) / (size * size)

    # Filtering process
    for c in range(image.shape[2]):
        for i in range(image.shape[0]):
            for j in range(image.shape[1]):
                region = padded[i:i+size, j:j+size, c]
                output[i, j, c] = np.sum(region
                                           * kernel)
            :
```

We implemented an averaging filter function *averaging_filter(image, size)* that applies a mean filter to each color channel of the input image.

A normalized kernel of size $k \times k$ is created using:

$$K = \frac{1}{k^2} \mathbf{1}_{k \times k}$$

Where all elements are equal and sum to 1, ensuring proper averaging. For each color channel c , the filtering operation is performed using three nested *for* loops that iterate over the image height, width, and channels.

Gaussian Filter

The Gaussian filter is a linear smoothing filter that reduces image noise and detail by convolving the image with a Gaussian kernel.

The kernel weights are derived from the 2D Gaussian function:

$$G(x,y) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$$

Here, σ controls the degree of smoothing; a larger σ results in stronger blurring. This filter is effective for reducing high-frequency noise while preserving overall image structure.

Unlike the averaging filter, it gives more weight to pixels near the center, which helps maintain edge transitions more naturally.

To construct the Gaussian kernel, we utilized *np.meshgrid* to generate a 2D grid of spatial coordinates. The kernel values were computed based on the Gaussian probability distribution using the provided size and standard deviation (std) parameters:

$$G(x,y) = \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$$

The kernel was subsequently normalized so that the sum of all weights equals 1.

For the filtering process, we implemented a set of nested for loops to apply the kernel to each spatial location and channel of the image, enabling per-pixel convolution across the RGB channels.

```
<Gaussian filter function>
def gaussian_filter(image, size, std):
    :
    # kernel generation
    k_x = np.arange(-pad, pad + 1)
    xx, yy = np.meshgrid(k_x, k_x)
    kernel = np.exp(-(xx**2+yy**2) / (2.* std**2))
    kernel = kernel / np.sum(kernel)
    # filtering process
    filtered = np.zeros_like(image, dtype=np.float
                                64)

    for y in range(H):
        for x in range(W):
            for c in range(C):
                region = padded[y:y+size, x:
                                    x+size, c]
                filtered[y, x, c] = np.sum(region
                                            * kernel)
    :
```

Bilateral Filter

The bilateral filter is a nonlinear, edge-preserving smoothing filter that replaces each pixel with a weighted average of its neighboring pixels, considering both spatial proximity and photometric similarity.

The filtered pixel value at location (x,y) is computed as: $I'(x,y) =$

$$\frac{1}{W_p} \sum_{(i,j) \in N(x,y)} I(i,j) G_s(|(i,j) - (x,y)|) G_r(|I(i,j) - I(x,y)|)$$

G_s is the spatial Gaussian, controlling the influence of nearby pixels based on their distance,

G_r is the range Gaussian, controlling the influence based on intensity (color) similarity,

W_p is a normalization factor ensuring that the weights sum to 1.

By jointly considering both spatial and range distances, the bilateral filter effectively smooths regions with similar colors while preserving sharp edges, making it particularly useful in tasks such as denoising, detail enhancement, and depth map refinement.

To implement edge-preserving smoothing, we designed a bilateral filter function that computes a weighted average of neighboring pixels based on both spatial proximity and color similarity. The function takes as input an RGB image and three parameters: size (the kernel size), sigma_color (range standard

deviation), and sigma_space (spatial standard deviation).

1. Spatial Gaussian Kernel generation

A fixed spatial kernel is computed using a 2D Gaussian distribution over the pixel coordinates:

$$G_s(x,y) = \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$$

A larger spatial sigma allows more distant pixels to contribute to the filtering.

2. Color Kernel generation

For each pixel location (x,y) and color channel c, a local region centered at (x,y) is extracted. The intensity difference between each pixel in the region and the center pixel is used to compute the range (color) kernel:

$$G_r(\Delta I) = \exp\left(-\frac{(I(i,j) - I(x,y))^2}{2\sigma^2}\right)$$

A larger range sigma enables pixels with greater color differences to influence the result. Weights are normalized such that they sum to 1, and the output value is obtained by weighted summation over the region. This implementation preserves image edges by suppressing smoothing across intensity boundaries, making it effective for denoising while maintaining structural detail.

```
<bilateral_filter function>
def bilateral_filter(image, size, sigma_color, sigma_sp
    ace):
    :
    # spatial Gaussian kernel coordinates
    ax = np.arange(-pad, pad + 1)
    # coordinate grid for the filter window
    xx, yy = np.meshgrid(ax, ax)
    # spatial kernel generation
    spatial_kernel = np.exp(-(xx**2 + yy**2) / (2 *
    sigma_space**2))

    for y in range(H):
        for x in range(W):
            for c in range(C):
                center_val = padded[y+pad, x+pad, c]
                region = padded[y:y+size, x:x+size, c]

    # color Gaussian: high values for similar colors
    color_diff = region - center_val
    color_kernel = np.exp(-(color_diff **
    2) / (2 * sigma_color ** 2))
    # final bilateral weights = spatial * color
    bilateral_weights = spatial_kernel *
    color_kernel
    bilateral_weights /= np.sum(bilateral
    _weights)
    output[y, x, c] = np.sum(region *
    bilateral_weights)
    :
```

3.2.2 result

Noised Images	Initial PSNR(dB)	Testing Filter(kernel size 3×3)			Max:	Min:	
		Median	Averaging	Gaussian	Bilateral C=30	Bilateral C=45	Bilateral C=75
Impulse (p=0.05)	18.20	43.92	26.19	26.19	19.24	20.64	22.33
Impulse (p=0.1)	15.17	39.19	23.44	23.44	15.26	16.64	18.93
Gaussian (std=10)	28.13	28.18	31.37	31.39	33.03	31.81	31.07
Gaussian (std=30)	18.86	19.08	26.86	26.86	22.25	25.90	27.97
Gaussian (std=50)	14.82	15.37	23.39	23.39	16.27	18.92	22.60

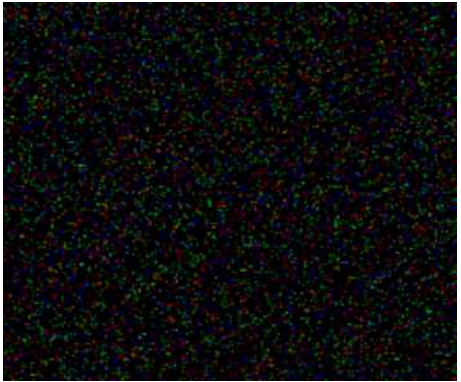
Table 2. PSNR of the restored image using different denoising filters with a kernel size of 3×3 . We set the standard deviation for the Gaussian and Bilateral filters to 15.

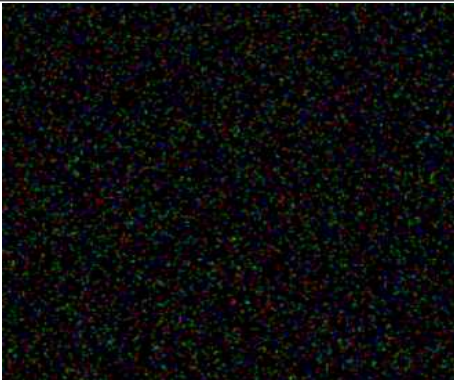
Noised Images	Initial PSNR(dB)	Testing Filter(kernel size 5×5)			Max:	Min:	
		Median	Averaging	Gaussian	Bilateral C=30	Bilateral C=45	Bilateral C=75
Impulse (p=0.05)	18.20	42.61	26.98	26.98	18.29	19.52	22.34
Impulse (p=0.1)	15.17	39.28	25.13	25.13	15.31	15.85	18.90
Gaussian (std=10)	28.13	28.17	28.99	29.00	32.75	31.81	30.55
Gaussian (std=30)	18.86	19.09	27.47	27.48	23.20	25.88	27.95
Gaussian (std=50)	14.82	15.42	25.38	25.38	16.83	18.93	22.63

Table 3. PSNR of the restored image using different denoising filters with a kernel size of 5×5 . We set the standard deviation for the Gaussian and Bilateral filters to 15,

Figure 3. noise = N, kernel size = 3×3 , spatial std = 15, color std = C, PSNR = P



Averaging, PSNR = 31.37			Gaussian, P = 31.39			Bilateral(C=75), P = 30.55		
								
Bilateral(C=60), P = 31.07			Bilateral(C=45), P = 31.81			Bilateral(C=30), P = 33.03		
								
2.N:Gaussian(std=30), P=18.86			original image			Median, P = 19.08		
								
Averaging, P = 26.86			Gaussian, P = 26.86			Averaging-Gaussian		
								

Bilateral(C=75), P = 27.97			Bilateral(C=45), P = 25.90			Bilateral(C=30), P = 22.25		
								
3.N:Gaussian(std=50), P=14.82			original image			Averaging, P = 23.39		
								
Gaussian, P= 23.39			Averaging-Gaussian			Bilateral(C=75) P = 22.60		
								
Bilateral(C=60), P = 20.97			Bilateral(C=45), P = 18.92			Bilateral(C=30), P = 16.27		
								

4.N:Impulse (p=0.05), P=18.20	original image	Median, P = 43.92
Averaging, P = 26.19	Gaussian, P = 26.19	Bilateral(C=75), P = 22.33
5.N:Impulse(p=0.1), P=15.17	original image	Median, P = 39.19
Averaging, P = 23.44	Gaussian, P = 23.44	Bilateral(C=75), P = 18.93

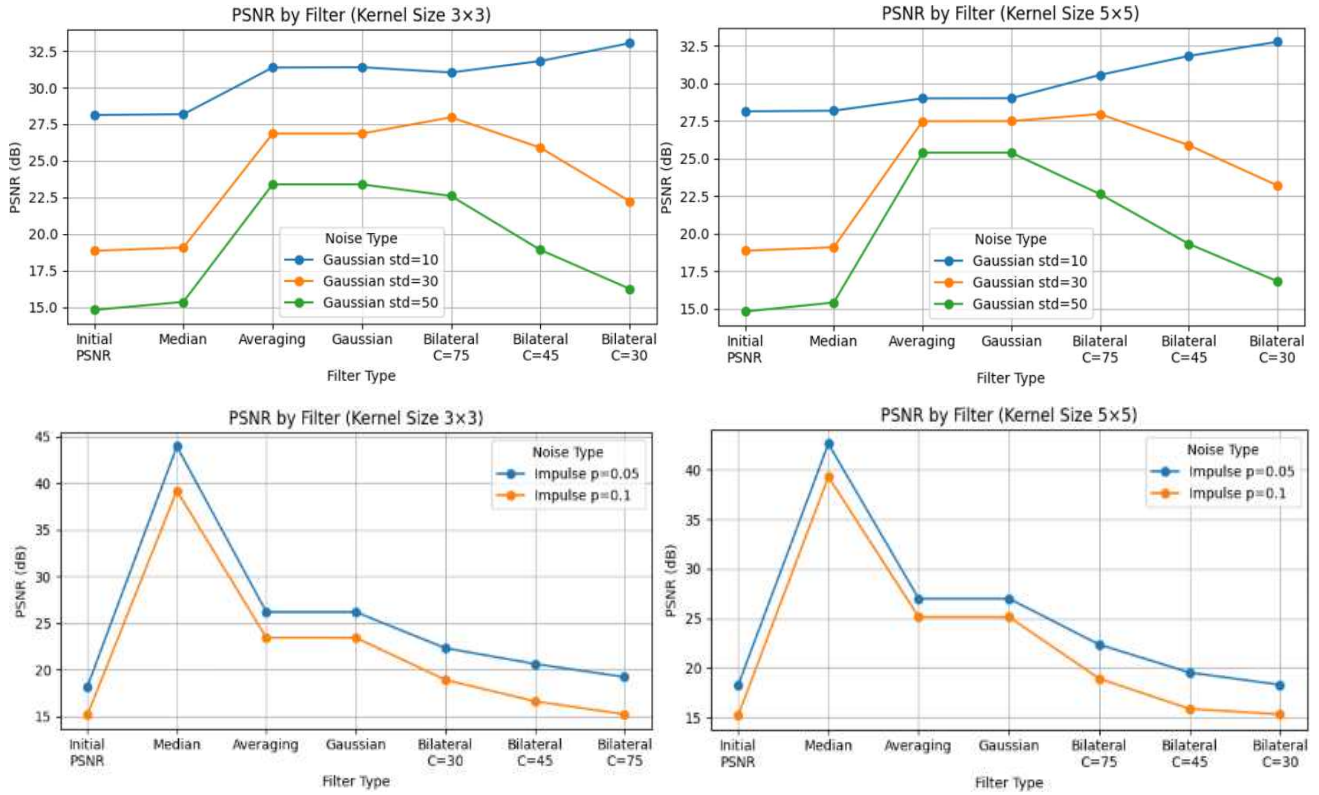


Figure 4. The graph presents PSNR values as a function of kernel size. To enhance clarity, results are separately illustrated for impulse-noised and Gaussian-noised images.

3.3 compare and analyze

We evaluated the performance of each filter under various conditions using Table 2, Table 3, and Figure 3, based on PSNR measurements. In particular, the graphs clearly show which filters are more robust against different types of noise. In addition to the numerical results, the filtered images are also presented to allow for subjective visual assessment.

Objective

To numerically evaluate the denoising performance of each filter, we compared the PSNR presented in Table 1 and Table 3, which report results using 3x3 and 5x5 kernel sizes, respectively. The filters were tested under varying noise conditions, including impulse noise (with probability values $p=0.05$ and $p=0.1$) and Gaussian noise (with standard deviations $\sigma=10, 30, 50$).

For impulse noise, the median filter consistently achieved the highest PSNR across all scenarios. Specifically, with $p=0.05$, the median filter produced a PSNR of 43.92dB (3x3 kernel) and 42.61dB (5x5 kernel), significantly outperforming other filters. Even under heavier impulse noise ($p=0.1$), the median filter remained the most effective, demonstrating its robustness to sparse, high-intensity noise.

In contrast, for Gaussian noise, bilateral filters exhibited superior performance. For Gaussian noise ($\sigma=10$), the bilateral filter with $C=30$ achieved the highest PSNR of 33.03dB for 3x3, 32.75dB for 5x5, outperforming standard Gaussian and averaging filters. As the noise intensity increased ($\sigma=30$ and $\sigma=50$), bilateral filters with higher C values ($C=75$) were more effective, maintaining higher PSNR compared to other filters, indicating their strong edge-preserving denoising capabilities.

Regarding kernel size, the performance difference between 3x3 and 5x5 filters was relatively minor. In many cases, the 3x3 kernel provided slightly better results, suggesting that increasing kernel size does not necessarily lead to improved PSNR.

In summary, the objective evaluation shows that the median filter is most suitable for removing impulse noise, while the bilateral filter is preferred for Gaussian noise. Kernel size had a limited impact, and optimal performance was primarily determined by the filter type and parameter tuning.

Subjective

In addition to the objective PSNR based analysis, we

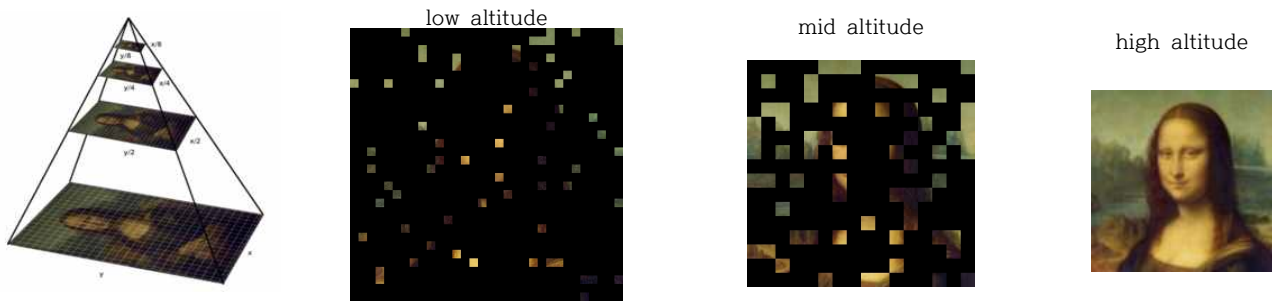


Figure 5. The main task in Problem 2 involves integrating images acquired from multiple altitudes to form a complete 512×512 image. Given the storage constraints of the UAV, the image acquired at high altitude (128×128) represents a complete but low-resolution view. In contrast, the mid-altitude image (256×256) contains only one-quarter of the full scene, while the low-altitude image (512×512) retains merely one-sixteenth of the original data.

conducted a subjective evaluation by visually comparing the filtered images. As shown in Figure 3, the median filter clearly preserves sharp edges and removes impulse noise effectively, resulting in clean and natural looking outputs. In contrast, the averaging and Gaussian filters tend to blur fine details, especially under high noise conditions, leading to a noticeable loss of structure. The bilateral filter, in some cases, makes the noise appear more noticeable in the visual output.

For Gaussian noise, bilateral filtering demonstrates superior perceptual quality. It successfully smooths out noise while maintaining edges and object boundaries, unlike the averaging filter which tends to over smooth the entire image. However, if the color sigma is not properly tuned according to the standard deviation of the noise, the bilateral filter may produce even stronger blurring than the Gaussian or averaging filters.

In the case of the given image, changes in kernel size produce negligible perceptual differences. In particular, variations in the spatial sigma of the bilateral and Gaussian filters result in minimal visual impact. Furthermore, a residual image between the Gaussian filtered and averaging filtered outputs reveals meaningful differences that are not immediately visible to the eye, as illustrated in Fig4.

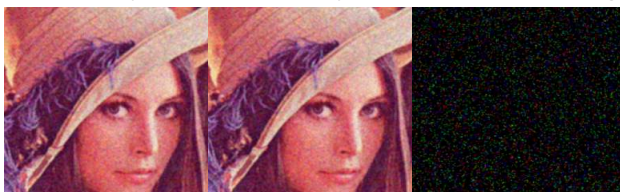


Figure 6. Left: Averaging filtered image, Center: Gaussian filtered image, Right: Residual between the two images. (Filtered images of a Gaussian noise-corrupted input.)

4. Problem 2: Image enhancement.

A planned UAV mission involves capturing the aerial image of a land area at different altitudes. The built-in camera in UAV is able to capture $K \times K$ pixel images. The mission begins with a 128×128 pixel global scene at the highest altitude, progresses to a 256×256 pixel image at a mid-altitude, and concludes with a 512×512 pixel image at the lowest altitude.

The challenge is that the UAV has limited memory ($128 \times 128 = 2^{14}$), requiring it to selectively choose sensing regions.

Let's develop a simulation platform to test the described mission scenario. Set $K = 16$.

1) First, apply the random region selection scheme. Then, use bilinear interpolation method to obtain the final 512×512 images. Check its performance in terms of PSNR.

-5.1.2 Performance

2) Build your own interpolation method to improve the performance.

-5.2 Build your own interpolation method

3) Build your own sensing region selection method to improve the performance

-5.3 Build your own sensing region method

4) Repeat the problem using various $K \in \{1, 2, 4, 8, \dots\}$ values.

-5.4 Repeat the problem using various K

5. Method: Image enhancement

5.1.1 Obtaining the final images.

Reconstructing a complete 512×512 image involves several stages. Initially, a downsampling module is required to emulate how the scene is perceived at each of the three altitude levels. Subsequently, a

cropping mechanism must be implemented to reflect the UAV's acquisition method, where only partial regions of the image are captured. Ultimately, an upsampling and reconstruction algorithm is necessary to generate the full-resolution image from the limited observations.

Image preparation

To simulate UAV observations at three altitude levels, we use a 512×512 base image. The low-altitude view uses the original image, the mid-altitude view is generated via one stage of downsampling (256×256), and the high-altitude view is generated through two stages of downsampling (128×128).

To simulate observations at different altitudes, we design a downsampling function that applies Gaussian blurring followed by spatial resolution reduction. A 2D Gaussian kernel is generated based on a given kernel size and standard deviation. The kernel is constructed using the Gaussian function, with coordinate grids generated by *np.meshgrid*, and then normalized to ensure a unit sum.

The image is padded using reflective padding to handle border regions, and convolution with the Gaussian kernel is performed for each channel independently, resulting in a blurred image. To reduce the resolution by a factor of two, we apply 2×2 average pooling. Specifically, the blurred image is reshaped into non-overlapping 2×2 blocks, and the mean of each block is computed to form the final downsampled output.

<Downsampling function>

```
def downsampling(image, size, sigma):
    :
    # kernel generation
    ax = np.arange(-size//2, size//2 + 1)
    xx, yy = np.meshgrid(ax, ax)
    kernel = np.exp(-(xx**2 + yy**2) / (2. * sigma**2))
    kernel = kernel / np.sum(kernel)
    # blurring
    k = kernel.shape[0]
    blurred = np.zeros_like(image, dtype=np.float32)
    for y in range(H):
        for x in range(W):
            for c in range(C):
                region = padded_img[y:y+k, x:x+k, c]
                blurred[y, x, c] = np.sum(region * kernel)
    # averaging downsampling
    H, W, C = blurred.shape
    blurred = blurred[:, H//2, W//2, :]
    downsampled = blurred.reshape(H//2, 2, W//2, 2, C).mean(axis=(1, 3))
    :
```

This process ensures that high-frequency details are suppressed before resolution reduction, preventing

aliasing artifacts.

Random Region Selection

Given the UAV's storage limitations, the final image budget is constrained to 128×128 pixels, independent of the number of captures. The parameter K controls the spatial resolution of each individual shot. When K=16, the UAV is able to take 64 separate images per altitude.

This configuration enables full-scene coverage at high altitude, partial coverage (1/4) at mid-altitude, and highly localized coverage (1/16) at low altitude. To simulate the UAV's limited acquisition capacity, the image is first divided into non-overlapping K×K patches. Among the total available patches, 64 are randomly selected to represent the regions captured by the UAV. A new image is generated by copying only the selected patches from the original image, while setting the rest to zero.

<Random choose function>

```
def random_choose(image, patch_size):
    :
    n_h = H // patch_size
    n_w = W // patch_size
    total_patches = n_h * n_w
    k = 2**14 // patch_size**2
    # choose random
    patch_indices = [(i, j) for i in range(n_h) for j in range(n_w)]
    selected = np.random.choice(len(patch_indices), 64, replace=False)
    selected_patches = [patch_indices[i] for i in selected]
    # output image
    output = np.zeros_like(image)
    for i, j in selected_patches:
        y, x = i * patch_size, j * patch_size
        output[y:y+patch_size, x:x+patch_size, :] = image[y:y+patch_size, x:x+patch_size, :]
    return output
    :
```

Upsampling and Restoration

The UAV reconstructs the final 512×512 image using the 64 images captured at each altitude level. We use Bilinear interpolation during the upsampling process. This method performs linear interpolation independently along both the horizontal x and vertical y axes, thereby computing a weighted average based on the values of the four nearest neighboring pixels. Given a target point (x, y) that lies between four known pixel values $f(x_0, y_0)$, $f(x_0, y_1)$, $f(x_1, y_0)$ and $f(x_1, y_1)$, the interpolated $f(x, y)$ is computed as:

$$f(x, y) = (1-dx)(1-dy)f(x_0, y_0) + dx(1-dy)f(x_1, y_0) + (1-dx)dyf(x_0, y_1) + dx dy f(x_1, y_1)$$

where $dx = x - x_0$, $dy = y - y_0$.

This approach ensures a smooth transition of intensity values across neighboring pixels and avoids abrupt artifacts, although it may introduce slight blurring, particularly in regions with high-frequency details. Bilinear interpolation is computationally efficient and widely adopted in practical applications such as image resizing, rotation, and warping, due to its balance between speed and visual quality.

Our image recovery strategy involves the following steps. Step1, the high altitude image is upsampled and used to fill the missing regions of the mid altitude image, resulting in a complete 256×256 image. Step2, this intermediate result is then upsampled and merged with the available portions of the low-altitude image to reconstruct the final 512×512 output.

<Upsampling and restoration functions>

```
def bilinear(image):
    :
    # bilinear interpolation
    for y in range(new_H):
        for x in range(new_W):
            # original position calculation
            src_y = y / 2.0
            src_x = x / 2.0

            # four adjacent pixel coordinates
            y0 = int(np.floor(src_y))
            x0 = int(np.floor(src_x))
            y1 = min(y0 + 1, H - 1)
            x1 = min(x0 + 1, W - 1)

            dy = src_y - y0
            dx = src_x - x0

            for c in range(C):
                top = (1 - dx) * image[y0, x0, c] + dx
                    * image[y0, x1, c]
                bottom = (1 - dx) * image[y1, x0, c]
                    + dx * image[y1, x1, c]
                value = (1 - dy) * top + dy * bottom
                upsampled[y, x, c] = np.clip(value, 0,
                    255)
            :
def restoration(image1, image2):
    # unsampling
    image1_up = bilinear(image1)
        .astype(np.uint8)

    # masking
    mask = np.all(image2 == 0, axis=-1)
    # merge
    output = image2.copy()
    output[mask] = image1_up[mask]
    :
```

The *bilinear_upsampling(image)* function increases the resolution of the input image by a factor of two using bilinear interpolation. For each pixel in the upsampled image, the corresponding location in the original image is calculated, and the value is estimated based on the four surrounding pixels. This process is repeated independently for each color channel.

The *restoration(image1, image2)* function merges two images of the same size: an upsampled image (image1) and a partially observed image (image2). A binary mask is created to detect pixels in image2 that are completely missing. The missing regions are then filled using the corresponding pixels from image1, while the observed regions in image2 are preserved.

5.1.2 Performance

As previously mentioned, the final image is reconstructed in two steps.

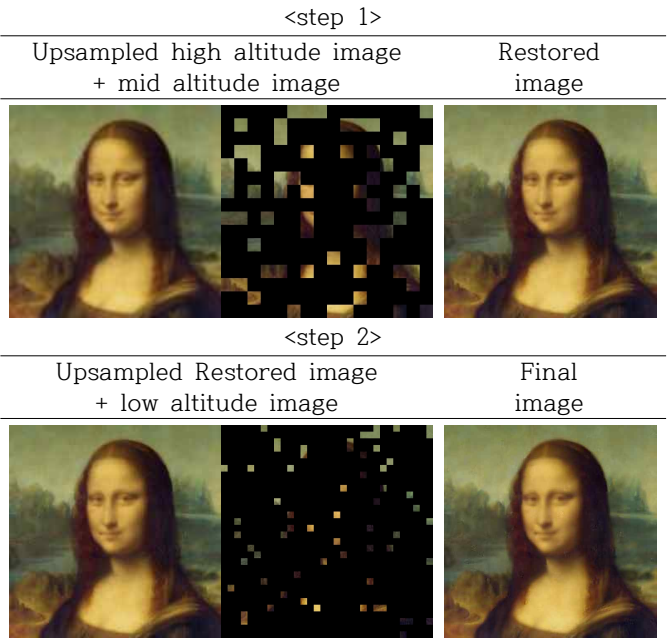


Figure 7. Restoration with two steps

Performance		
step	step 1	step 2
PSNR	37.15	32.85

Table 4. Performance in terms of PSNR

The PSNR of the final reconstructed image was measured at 32.85, indicating a high level of fidelity. While a reduction of approximately 5 dB was observed between Step 1 and Step 2, this drop is negligible in the context of the given experimental conditions.

5.2 Build your own interpolation method

As specified in the project guidelines, bilinear interpolation was employed during the upsampling stage. Although this method yields satisfactory results, its performance can be further improved. In this work, we enhance the reconstruction quality by incorporating bicubic interpolation and a forward interpolation strategy.

5.2.1 Ours1: Bicubic interpolation

Bicubic interpolation is an advanced image resampling method that improves upon bilinear interpolation by considering a larger neighborhood of pixels to estimate unknown values. Specifically, it uses the 16 nearest pixels surrounding the target coordinate and applies cubic convolution along both the horizontal and vertical directions. Bicubic interpolation is typically performed in two stages:

1. Cubic interpolation along x-axis:

$$f_y(u) = \sum_{k=-1}^2 a_k \cdot f(i+k, j)$$

$u = x - [x]$ and a_k : weights computed based on the
 cubic convolution kernel
 $f(i,j)$: Original pixel value

2. Cubic interpolation along y-axis:

$$f(x,y) = \sum_{l=-1}^2 b_l \cdot f_y(j+l)$$

$v=y-[y]$ and b_l : the vertical weights.

$f(x,y)$: Interpolated value

A common cubic kernel used in practice is based on Keys' function:

$$h(t) \begin{cases} (1.5|t|^3 - 2.5|t|^2 + 1), & 0 \leq |t| < 1 \\ (-0.5|t|^3 + 2.5|t|^2 - 4|t| + 2), & 1 \leq |t| < 2 \\ 0, & \text{otherwise} \end{cases}$$

t : distance between x and i

$h(t)$: Kernel function

The `bicubic_upsampling()` function enlarges an image by a factor of 2 using bicubic interpolation. For each pixel in the output image, its corresponding location in the input image is computed and a 4x4 neighborhood is used to estimate the new pixel value.

A *cubic weighting()* function is applied to compute the contribution of each neighboring pixel based on its distance from the target location. The weights along both axes are combined, and the final value is computed as a weighted sum of the 16 surrounding pixels. This process is repeated for each color channel.

This method offers smoother and sharper results

than simpler approaches like bilinear interpolation.

<bicubic interpolation functions>

```
def cubic_weight(t):
    t = abs(t)
    if t <= 1:
        return (1.5 * t - 2.5) * t * t + 1
    elif t < 2:
        return ((-0.5 * t + 2.5)*t-4)*t+2
    else:
        return 0

def bicubic_upsampling(image):
    :
    for y in range(new_H):
        for x in range(new_W):
            src_y = y / 2.0
            src_x = x / 2.0

            y_int = int(np.floor(src_y))
            x_int = int(np.floor(src_x))

            for c in range(C):
                value = 0.0
                for m in range(-1, 3):
                    for n in range(-1, 3):
                        yy = np.clip(y_int+m,0,H-1)
                        xx = np.clip(x_int + n, 0, W -1)
                        wy = cubic_weight(src_y-(y_int+m))
                        wx = cubic_weight(src_x-(x_int+n))
                        value+=image[yy, xx, c] *wy*wx
                    :

```

5.2.2 Ours2: Forward interpolation

target image	spread target	merge with next image
		

Figure8. Our method is based on forward interpolation with both low and high resolution images.

While forward interpolation typically underperforms compared to backward interpolation, the current setting goes beyond standard upsampling. The reconstruction process utilizes two resolutions simultaneously. As shown in Figure 8, we first spatially spread the low-resolution image, fill only the missing regions with high-resolution data, and subsequently apply forward interpolation.

<forward interpolation function>

```
def expand_sparse_grid(image):
    :
    img_sparse = np.zeros((H * 2, W * 2, C), dtype=
                        image.dtype)
    img_sparse[:, ::2] = image
    return img_sparse

def image_assemble(image1, img_sparse):
    :
    mask = np.all(image1 == 0, axis=2)
    result = image1.copy()
    result[mask] = img_sparse[mask]
    :

def bilinear_fill_sparse_numpy(image):
    :
    if not mask[y, x]:
        vals = []
        weights = []

        for dy in [-1, 0, 1]:
            for dx in [-1, 0, 1]:
                ny, nx = y + dy, x + dx
                if (0 <= ny < H) and (0 <= nx < W) and m
                    ask[ny, nx]:
                    dist = max(abs(dy), abs(dx))
                    weight = 1.0 / (dist + 1e-5)
                    :
                    :
```

The *expand_sparse_grid* function generates a sparse high-resolution image by placing the pixels of a low-resolution image at every other location in a grid that is **twice the spatial resolution**. All intermediate pixels are initialized as zero.

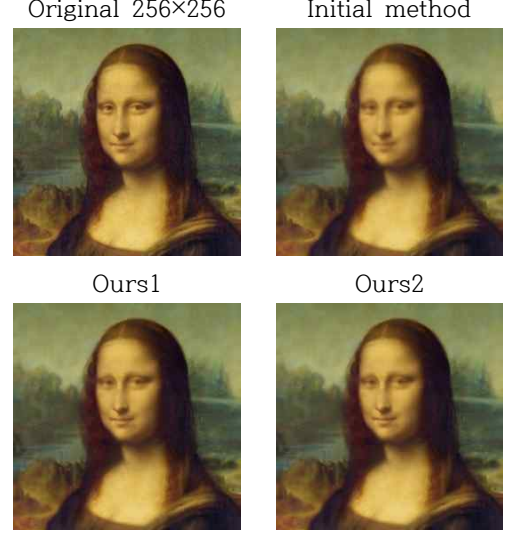
The *image_assemble* function merges the high-resolution image which have missing regions with the sparse image produced in the previous step. A binary mask is created by identifying locations in the high-resolution image that are empty. These missing regions are selectively filled with the corresponding pixels from the sparse image.

The *bilinear_fill_sparse_numpy* function fills the remaining unpopulated pixels in the image using weighted averages of nearby known pixels. For each empty pixel, the algorithm collects valid neighboring values within a 3×3 window and applies a distance-based inverse weighting scheme. (Unlike a standard averaging filter, this method excludes neighboring pixels if their values are zero across all channels.)

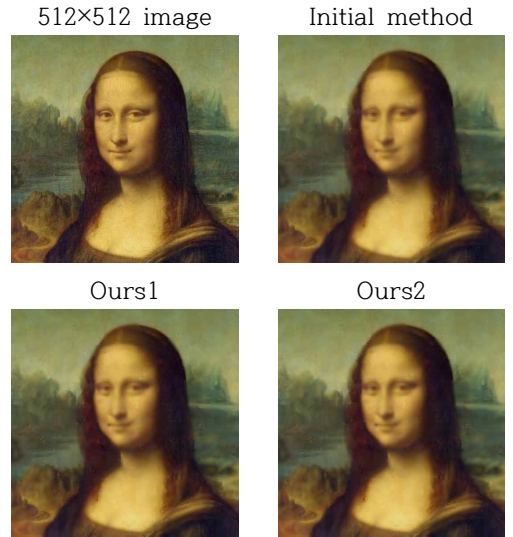
5.2.3 Performance

The proposed methods, Ours1 and Ours2, demonstrate improved reconstruction quality over the Initial method. In Step1 (256×256), Ours1 achieved the highest PSNR of 37.55 dB, outperforming the Initial method's 37.15 dB. Ours2 also showed competitive performance with 37.40 dB. In Step2 (512×512), Ours2

yielded the best result at 32.99 dB, slightly higher than the Initial method's 32.85 dB, while Ours1 recorded 32.68 dB. Visual inspection confirms that the proposed methods preserve finer image details more effectively. These results validate the effectiveness of our approach across different resolutions.



Step1	Method		
PSNR(dB)	Initial	Ours1	Ours2
	37.15	37.55	37.40



Step2	Method		
PSNR(dB)	Initial	Ours1	Ours2
	32.85	32.88	32.89

Figure 9. PSNR Comparison Between Project Method and Proposed Method

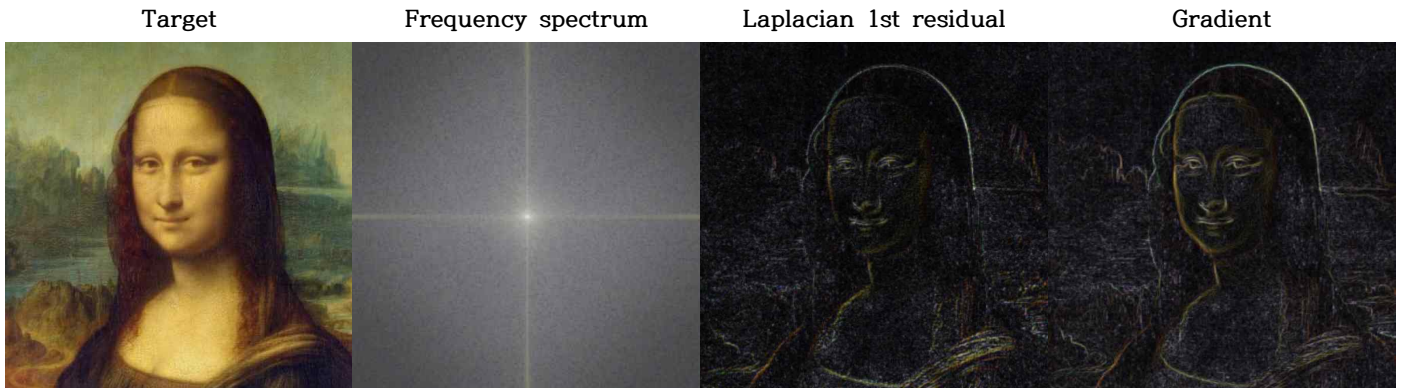


Figure 10. We analyze the image by transforming it into the frequency spectrum, Laplacian residual map, and gradient map in order to identify major regions.

5.3 Build your own sensing region method

In the image reconstruction process, not only interpolation but also region selection plays a crucial role in image enhancement. Section 5.3 describes our strategy for region selection in detail. In real-world sensing systems, low-resolution observations are commonly utilized to guide the selection of regions for high-resolution sensing, enabling efficient allocation of sensing resources. Following this principle, our method first leverages high-latitude (coarse-resolution) images to identify informative regions in the mid-latitude image, and subsequently determines which specific areas of the low-latitude (fine-resolution) image should be sensed.

In order to select the most informative blocks for image reconstruction, we considered **three complementary criteria**. For each criterion, we applied **two evaluation strategies**, allowing for a comprehensive comparison of selection effectiveness. Each of these metrics captures high-frequency content from a different perspective.

Frequency domain energy directly reflects the presence of rapid intensity changes across the entire block, providing a global measure of texture or fine detail.

Laplacian pyramid 1st residuals capture the high-frequency components lost during Gaussian downsampling, highlighting the structural details that are most sensitive to resolution changes.

Gradient magnitude focuses on local spatial variation, emphasizing edges and transitions in intensity within each block.

In the frequency-based method, computations were performed on a grid-by-grid basis, where the image was divided into fixed-size blocks and frequency features were extracted from each grid independently. In contrast, both the Laplacian pyramid and gradient-based approaches computed feature

responses over the entire image first, and then selected informative grids based on the resulting global feature maps.

5.3.1 Frequency domain energy

<Frequency choose function>

```
def fft_spectrum_energy(patch, ratio=0.25):
    f = np.fft.fft2(patch)
    fshift = np.fft.fftshift(f)
    magnitude = np.abs(fshift)
    H, W = magnitude.shape
    cy, cx = H // 2, W // 2

    low = int(min(H, W) * ratio / 2)
    mask = np.ones_like(magnitude, dtype=bool)
    mask[cy - low:cy + low, cx - low:cx + low] = False

    high_freq_energy = np.std(magnitude[mask])
    #high_freq_energy = np.sum(magnitude[mask])
    return high_freq_energy

def frequency_choose(image, target, grid_size):
    energies, positions = [], []
    k = 2**14 // (grid_size * grid_size)
    for y in range(0, H, grid_size):
        for x in range(0, W, grid_size):
            patch = gray_image[y:y+grid_size, x:x+grid_size]

            energy = fft_spectrum_energy(patch)
            energies.append(energy)
            positions.append((y, x))

    energies = np.array(energies)
    positions = np.array(positions)

    top_idx = np.argsort(energies)[-k:]
    masked_image = np.zeros_like(image)

    for idx in top_idx:
        y, x = positions[idx]
        masked_image[y:y+grid_size, x:x+grid_size] =
        target[y:y+grid_size, x:x+grid_size]
```

To identify regions with high structural detail, we introduce a frequency-based patch selection method grounded in spectral energy analysis. The input image is first converted to grayscale and partitioned into non-overlapping grids of size $G \times G$. For each patch P , we compute its two-dimensional Fast Fourier Transform (FFT), denoted as $F(P)$, and apply a frequency shift so that the zero-frequency component is centered. We then define a high-frequency mask that excludes a square region around the center of the spectrum, whose size is controlled by a ratio parameter ρ . Specifically, we suppress the central $2(\rho \cdot \min(H, W))^2$ region of the spectrum to focus on high-frequency components:

$$h(t) = \begin{cases} 0, & \text{if } (u, v) \in \text{center region of size } \rho \times \min(H, W) \\ 1 & \text{otherwise} \end{cases}$$

The spectral energy of each patch is defined as the standard deviation(or sum) of the magnitude spectrum within the high-frequency region:

$$E(P) = \text{std}(|F(P)|_{\text{high frequency}})$$

After computing energies for all patches, we select the top- k patches with the highest spectral energy, where $k=2^{14}/G^2$. This ensures a constant number of informative regions are selected regardless of grid resolution. The final output is generated by retaining the corresponding regions from a reference or target image, effectively masking less informative areas and preserving high-detail regions.

5.3.2 Laplacian pyramid 1st residual

<L1 choose function>

```
def laplacian_choose(image, target, grid_size):
    :
    L1 = np.abs(image-image_prior)
    scores = []
    positions = []
    k = 2**14 // (grid_size * grid_size)
    for y in range(0, H, grid_size):
        for x in range(0, W, grid_size):
            patch=L1[y:y+grid_size, x:x+grid_size]
            score = np.std(patch)
            #score = np.sum(patch)
            scores.append(score)
            positions.append((y, x))

    scores = np.array(scores)
    positions = np.array(positions)
    :
```

To capture regions of local detail and structural variation, we employ a Laplacian-based scoring strategy grounded in inter-scale image difference. Given an input image I and its corresponding Laplacian pyramid approximation I_{prior} , we first com-

pute the absolute difference map $L_1 = |I - I_{prior}|$, which emphasizes edges and fine structures suppressed in the low-pass representation.

The image is then partitioned into non-overlapping grids of size $G \times G$, and for each patch, a local score is computed based on the statistical variation within the difference map:

$$S(P) = \text{std}(L_1[P])$$

where $\text{std}(\cdot)$ denotes the standard deviation operator and P represents the grid region. Optionally, the sum value can be used as an alternative metric.

After computing scores for all patches, we select the top- k grids, where $k=2^{14}/G^2$, ensuring consistency in selection quantity across resolutions. These high score patches indicate regions with rich detail or edges, and are used to selectively reconstruct or refine the target image by copying only the corresponding high-scoring regions into a blank canvas.

5.3.3 Gradient magnitude

<Gradient choose function>

```
def gradient_choose(image, target, grid_size):
    :
    gray_padded = np.pad(gray, pad_width=1, mode='
                                reflect')
    gx = np.abs(gray_padded[1:-1, 2:] - gray_padded
                                [1:-1, 1:-1])
    gy = np.abs(gray_padded[2:, 1:-1] - gray_padded
                                [1:-1, 1:-1])
    grad_mag = np.sqrt(gx**2 + gy**2) # (H, W)
    gradients = []
    positions = []
    k = 2**14 // (grid_size * grid_size)

    for y in range(0, H, grid_size):
        for x in range(0, W, grid_size):
            patch_grad = grad_mag[y:y+grid_size, x:x
                                +grid_size]

            #std_grad = np.std(patch_grad)
            #sum_grad = np.sum(patch_grad)
            gradients.append(std_grad)
            positions.append((y, x))
    gradients = np.array(gradients)
    positions = np.array(positions)
    :
```

To identify image regions with strong edge or texture content, we utilize a gradient magnitude-based selection strategy. The input image is first converted to grayscale, and gradient maps are computed in the horizontal and vertical directions using a first-order finite difference approximation. Specifically, we define the directional gradients G_x and G_y as follows:

$$G_x = |I(x+1, y) - I(x, y)|, G_y = |I(x, y+1) - I(x, y)|$$

Padding is applied to the grayscale image to ensure valid boundary computation. The gradient magnitude

map is then computed as:

$$G = \sqrt{G_x^2 + G_y^2}$$

This global gradient map reflects the strength of local variations and edges across the image. The image is then partitioned into non-overlapping grids of size $G \times G$, and the std gradient magnitude within each patch is computed:

$$S(P) = \frac{1}{|P|} \sum_{(x,y) \in P} G(x,y)$$

we sort the patches by their scores and select the top k regions, where $k=2^{14}/G^2$, to ensure a consistent total area coverage. The selected patches correspond to regions with high structural complexity, which are then used to construct a sparse representation by copying the corresponding blocks from a reference or target image.

5.3.4 Performance

In accordance with the guideline, we set the grid size to 16×16 . We evaluated each patch using two different scoring methods: **sum** and **standard deviation**. (Since all patches contain the same number of pixels, the sum and mean produce equivalent results.)

Method		STEP 1	STEP 2
S U M	Baseline	37.15	32.85
	Frequency	38.69	33.71
	GL L1	38.26	33.38
	Gradient	39.29	34.12
S T D	Frequency	38.55	33.67
	GL L1	38.66	33.61
	Gradient	39.27	34.06

Table 5. PSNR, Grid size 16×16

The experimental results are summarized in Table 5. The baseline PSNR results for STEP 1 and STEP 2 were 37.15 dB and 32.85 dB, respectively. Among the tested scoring strategies, the gradient-based method outperformed others consistently across both steps and both scoring metrics.

When using the sum as the scoring function, gradient selection achieved the highest PSNR of 39.29 dB in STEP 1 and 34.12 dB in STEP 2, surpassing all other configurations. The frequency-based method also showed improved performance over the baseline but was consistently lower than the gradient-based counterpart.

Similarly, under the standard deviation metric, the gradient-based approach again yielded the best results with 39.27 dB in STEP 1 and 34.06 dB in STEP 2. These findings demonstrate that gradient magnitude is a more effective indicator of informative content

compared to frequency energy or Laplacian difference in the given image reconstruction task.

Overall, the results confirm that gradient magnitude based region selection.

Effect of Scoring Metrics: Sum vs. Standard Deviation

While both sum and standard deviation serve as plausible scoring metrics for patch selection, their underlying characteristics lead to subtle yet meaningful differences in performance.

The sum based scoring primarily reflects the overall intensity or magnitude of the signal within a patch. In the context of gradient magnitude or Laplacian difference maps, higher sum values typically correspond to regions with more edge content or structural detail. However, this approach may overemphasize large homogeneous regions with uniformly high values, such as broad textures or repetitive patterns, which do not always contribute critically to perceptual quality.

In contrast, the standard deviation-based scoring captures local contrast and variation within each patch. It tends to highlight regions where intensity or structure changes sharply, such as fine edges or boundaries between objects. These areas are more likely to be perceptually significant and beneficial for restoration tasks, especially in the context of multi-step reconstruction where subtle details are progressively refined.

This distinction explains why, in some cases (e.g., frequency with GL L1), the standard deviation metric outperforms sum based scoring in STEP2, where preserving fine structure becomes more critical. Conversely, in STEP1, sum based scoring occasionally performs better by prioritizing large, high-energy regions that dominate the early stage reconstruction.

5.3.5 Additional Experiments: Influence of Image Characteristics

To examine the generalizability of our method, we conducted additional experiments using images with varying color distributions. While the original test image (Mona Lisa) predominantly consists of greenish tones and smooth textures, which minimize the influence of color variation, alternative test images with more diverse and saturated colors were also evaluated.

The results revealed that images with greater color diversity and chromatic complexity tend to yield lower restoration performance, indicating that color variation introduces additional challenges in patch-based selection and reconstruction.

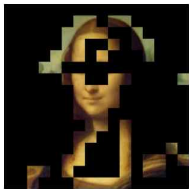
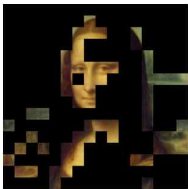
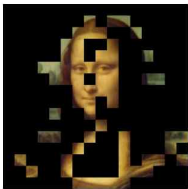
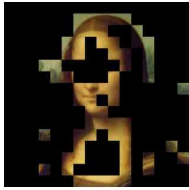
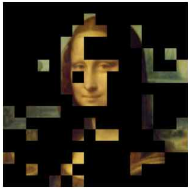
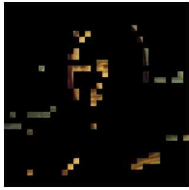
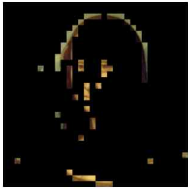
Region Selection Using Three Criteria and Two Evaluation Methods						
scoring method	Choose in 256×256 image			Choose region in 512×512 image		
	Frequency	GL L1	Gradient	Frequency	GL L1	Gradient
SUM						
STD						

Figure11. Both gradient and frequency-based approaches tended to identify similar regions, particularly focusing on fine details such as the eyes, nose, and mouth.

Restored image (Up:sum method, Down: std method)		
Frequency 512×512 image	GL L1 512×512 image	Gradient 512×512 image
		
		

Figure12. Despite the enhanced sharpness relative to the initial approach, locally blurred regions are still present and may draw visual attention.

region sensing: Gradient(sum) + interpolation: Ours2		
PSNR(dB)	Step1 39.36	Step2 34.19

Experiment with another image

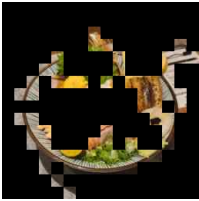
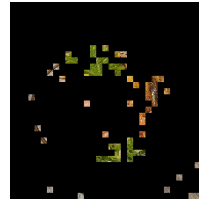
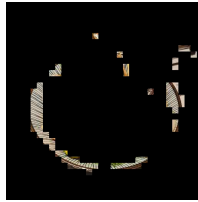
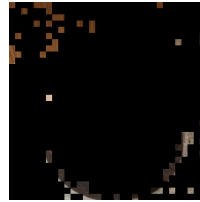
scoring method	Choose in 256×256 image			Choose region in 512×512 image		
	Frequency	GL L1	Gradient	Frequency	GL L1	Gradient
SUM						
	31.58	30.17	30.53	24.54	23.38	23.58
STD						
	31.09	28.00	30.11	24.37	22.10	23.53

Figure13. Region Selection Using Three Criteria and Two Evaluation Methods with another image

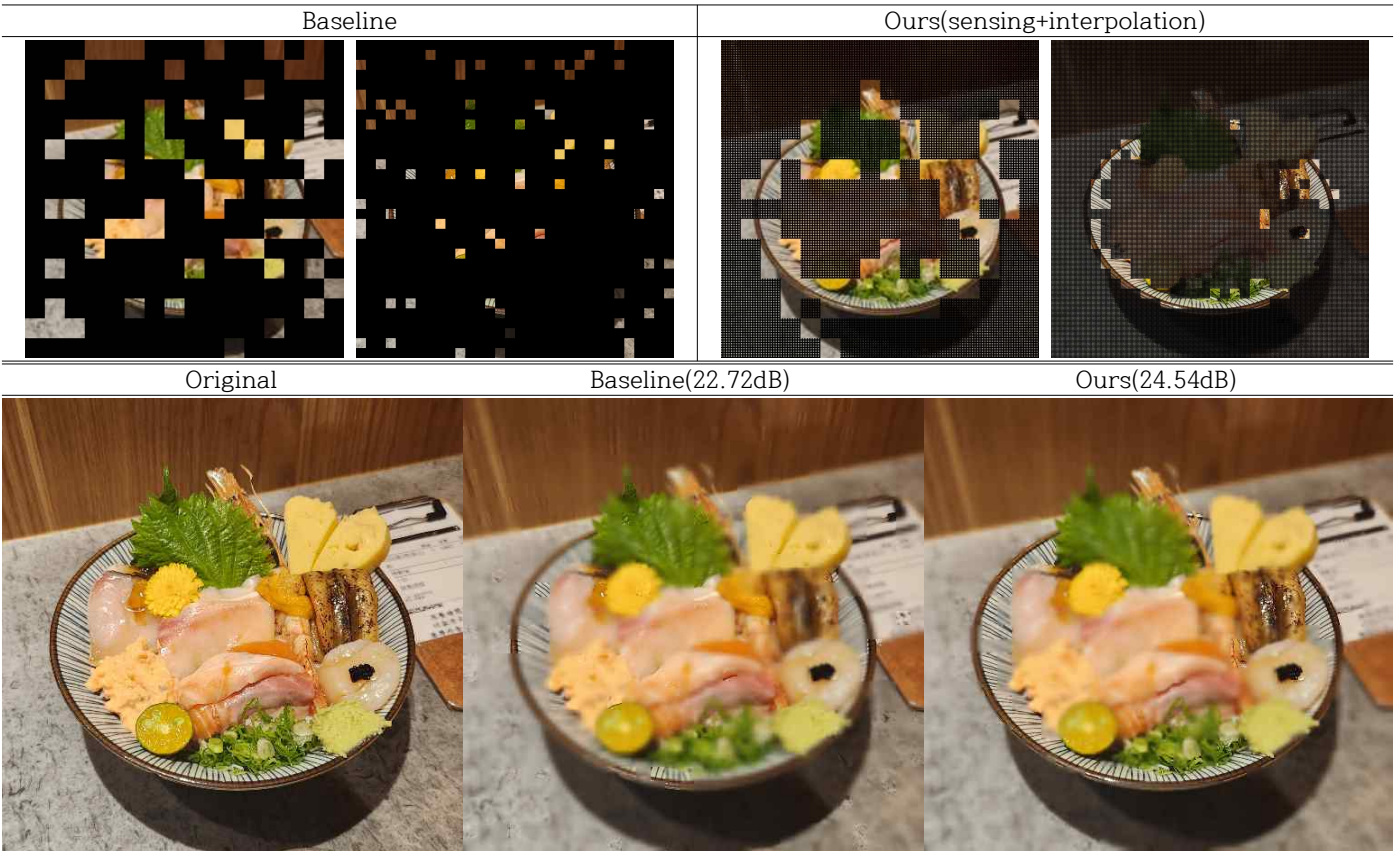


Figure14. For performance verification, an additional experiment was conducted on a different image. The best PSNR result, 24.54, was obtained using the frequency-based selection with sum as the scoring method.

5.4 Repeat the problem using various K

Method	Image	Kernel Size (PSNR)				
		K=1	K=2	K=4	K=8	K=16
Baseline sensing	256×256	37.30	37.24	37.21	37.20	37.15
Baseline interpol	512×512	33.05	32.97	32.93	32.89	32.85
Baseline sensing	256×256	38.06	37.50	37.42	37.40	37.27
our interpol2	512×512	33.96	33.38	33.18	33.07	32.98
our sensing	256×256	40.30	40.50	40.46	39.98	39.29
Baseline interpol	512×512	34.90	34.92	34.79	34.54	34.12
our sensing	256×256	40.54	40.49	40.47	40.02	39.36
our interpol	512×512	35.10	34.99	34.85	34.56	34.19

Table 6. This table presents PSNR results obtained using various kernel sizes and sensing & interpolation strategies.

Strategy

Table 6 presents the PSNR results for different image resolutions and kernel sizes, comparing baseline methods with our proposed sensing and interpolation strategies. The evaluation is conducted on two image resolutions: 256×256 and 512×512, with kernel sizes ranging from K=1 to K=16. In our framework, the sensing strategy is based on gradient magnitude analysis, while the interpolation process adopts the Our2 reconstruction method.

Comparison: Baseline vs. Ours

Across all kernel sizes and resolutions, our proposed methods consistently outperform the baseline in terms of PSNR. Notably, at 256×256 resolution, the our sensing method achieves a PSNR of 40.54 dB at K=1, significantly higher than the baseline sensing value of 37.30 dB under the same condition.

At 512×512 resolution, our sensing with our interpol method records the highest PSNR of 35.10 dB at K=1, outperforming the baseline method, which yields 33.05 dB.

Effect of Kernel Size

We observe a general downward trend in PSNR as K increases, which reflects the increasing difficulty of image reconstruction from sparser information. However, the rate of degradation differs notably between methods. Baseline methods show steady PSNR drops as K increases (e.g., 37.30 → 37.15 at 256×256).

Our sensing method, by contrast, maintains relatively stable performance, with only ~1.3 dB drop across K=1 to K=16 (40.30 → 39.26).

This robustness indicates that gradient-based patch prioritization effectively selects critical information regardless of kernel size.

Best and Worst Cases

Best Case: The highest PSNR (35.10 dB) is achieved by

our sensing at 512×512 resolution with K=1, indicating that a modest level of sampling combined with gradient guidance provides optimal reconstruction quality.

Worst Case: The lowest PSNR (32.85 dB) is observed in the baseline 512×512 method at K=16, showing significant degradation under sparse conditions without any informed patch selection.

This contrast of +2.25 dB between the best and worst cases reinforces the effectiveness of intelligent patch selection and the sensitivity of naive methods to sparsity.

Conclusion

From these experiments, we conclude that gradient-based sensing is consistently the most effective strategy across resolutions and kernel sizes, offering high PSNR and graceful degradation under sparsity.

Forward interpolation using the next image provides steady improvement over baseline interpolation, leveraging temporal coherence effectively.

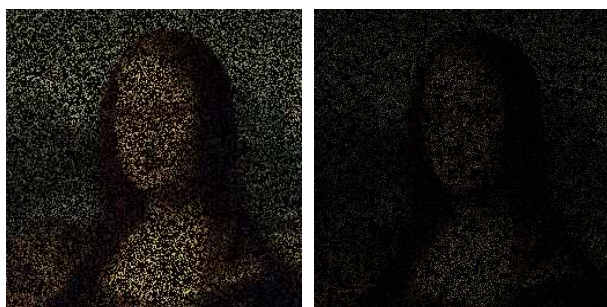
Kernel size (K) significantly affects performance, but the impact is substantially mitigated when content aware or predictive strategies are applied.

The choice of reconstruction strategy plays a larger role than sampling density alone, especially in low-data regimes.

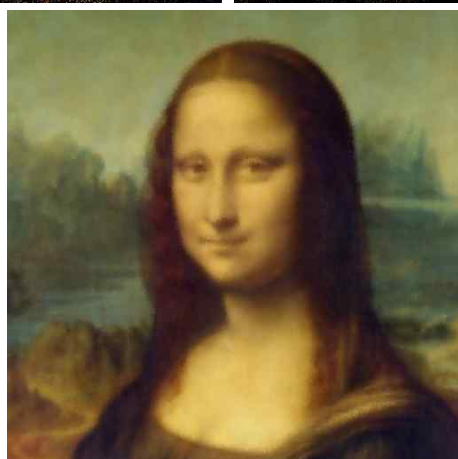
These findings validate our approach and support its applicability in practical low measurement image reconstruction scenarios.

kernel size = 1

Baseline

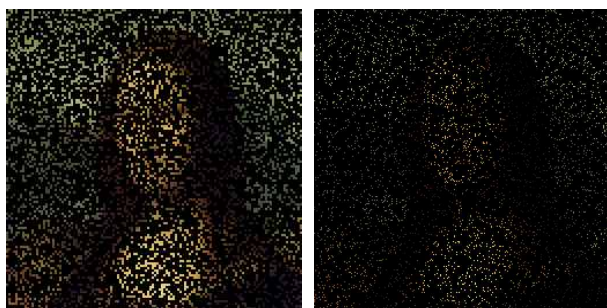


Ours(sensing + interpolation)

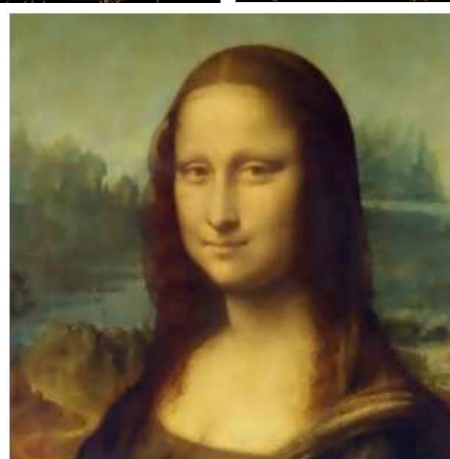
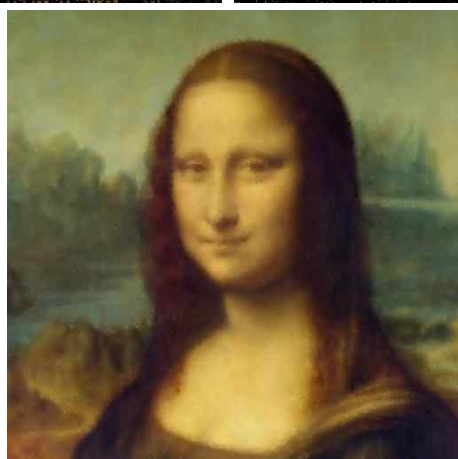


kernel size = 2

Baseline

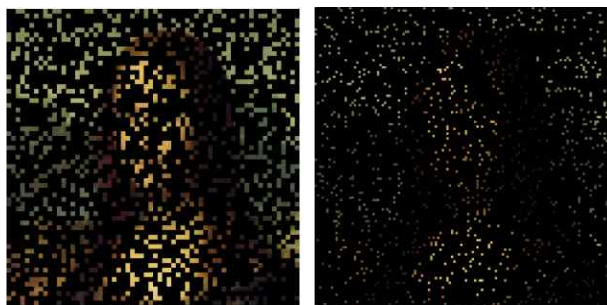


Ours(sensing + interpolation)

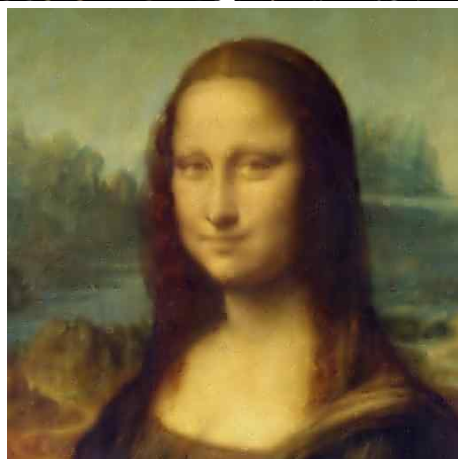


kernel size = 4

Baseline

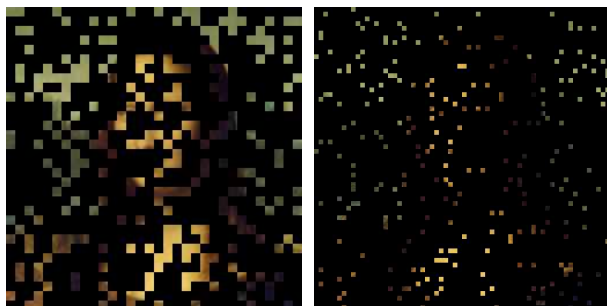


Ours(sensing + interpolation)



kernel size = 8

Baseline



Ours(sensing + interpolation)

